An Introduction to Reinforcement Learning

Shuaicheng Niu
niushuaicheng@gmail.com

South China University of Technology
Southern Artificial Intelligence Laboratory(SAIL)
Group of Reinforcement Learning

July 29, 2018

Outline

1. What is Reinforcement Learning?
2. Markov Decision Process
3. Value Based Methods
   - Dynamic Programming
   - Q-Learning
   - Value Function Approximation
4. Policy Gradient Methods
   - Likelihood Ratio Gradient
   - Variance Reduction
   - Actor Critic
5. Summary

Outline

1 What is Reinforcement Learning?

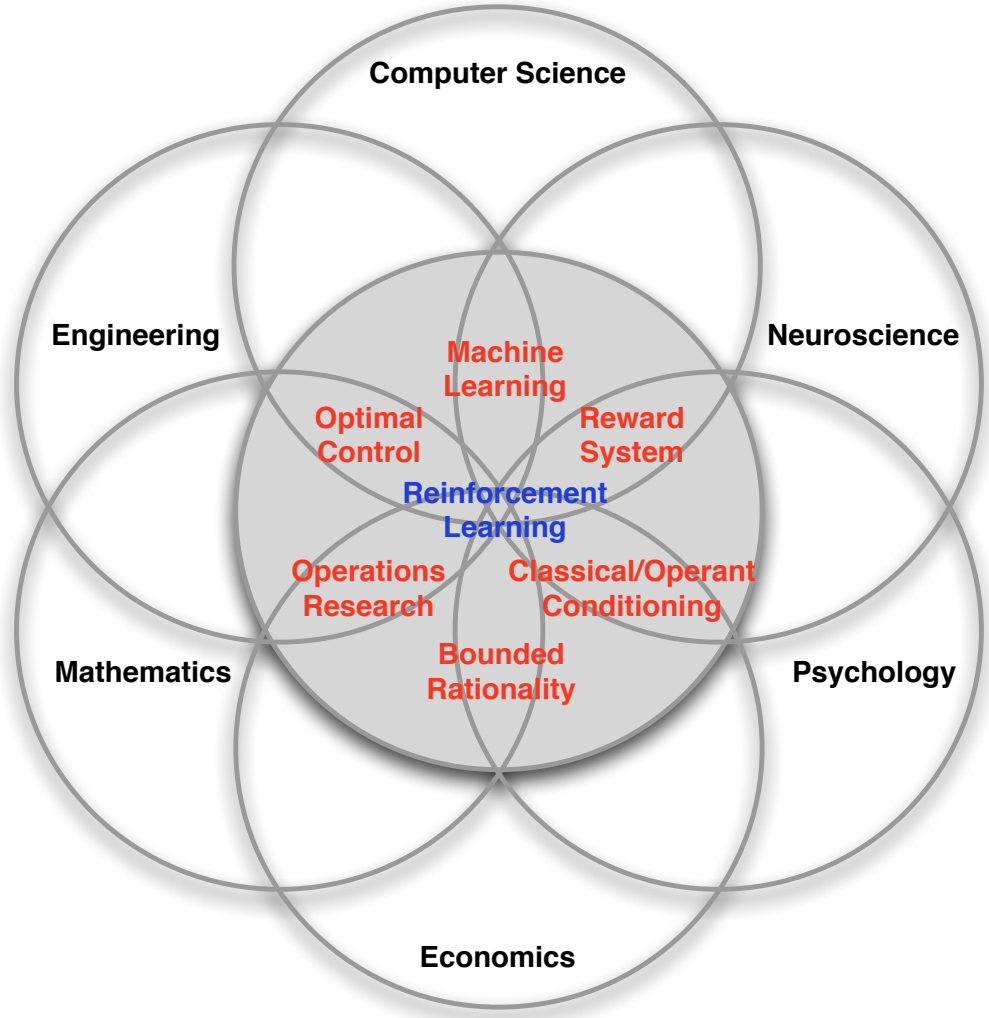
2 Markov Decision Process

3 Value Based Methods
- Dynamic Programming
- Q-Learning
- Value Function Approximation

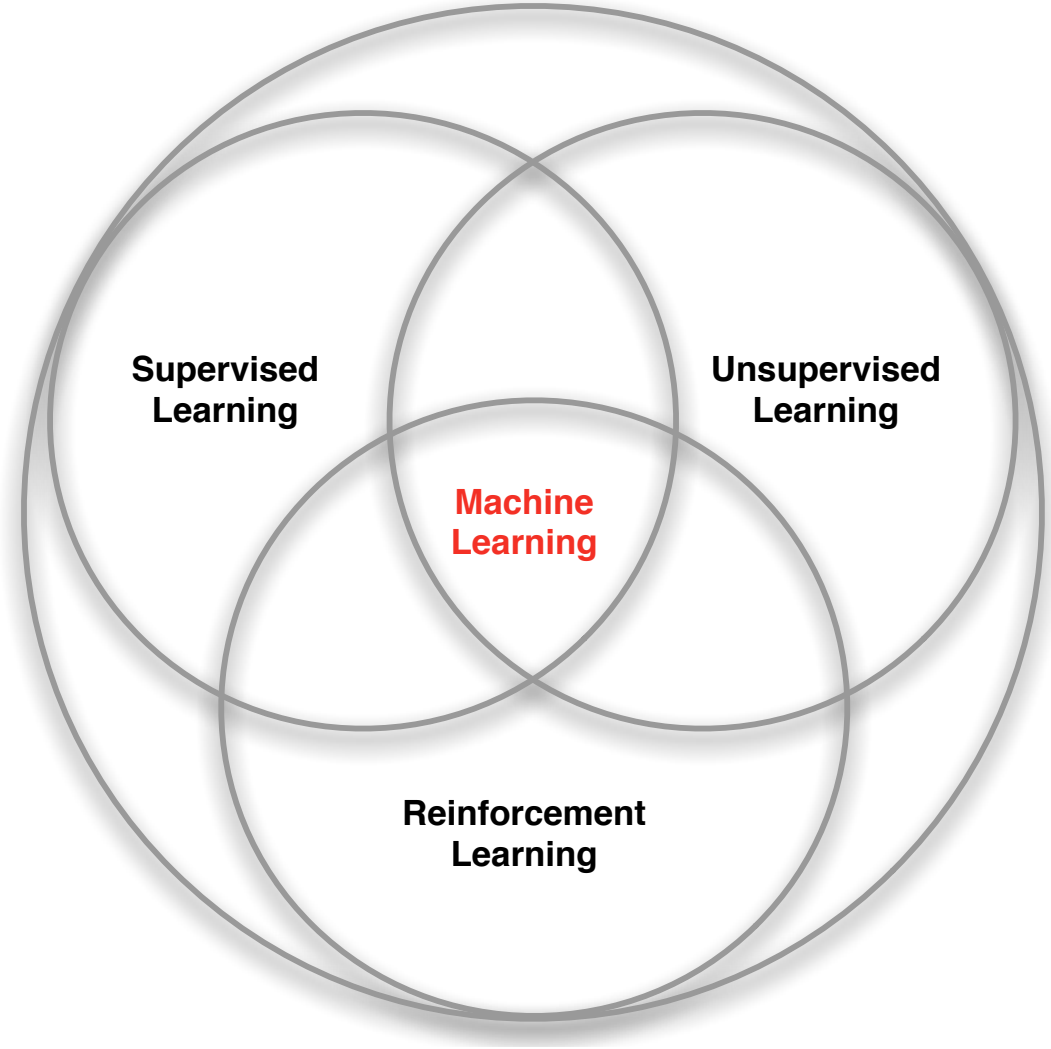
4 Policy Gradient Methods
- Likelihood Ratio Gradient
- Variance Reduction
- Actor Critic

5 Summary

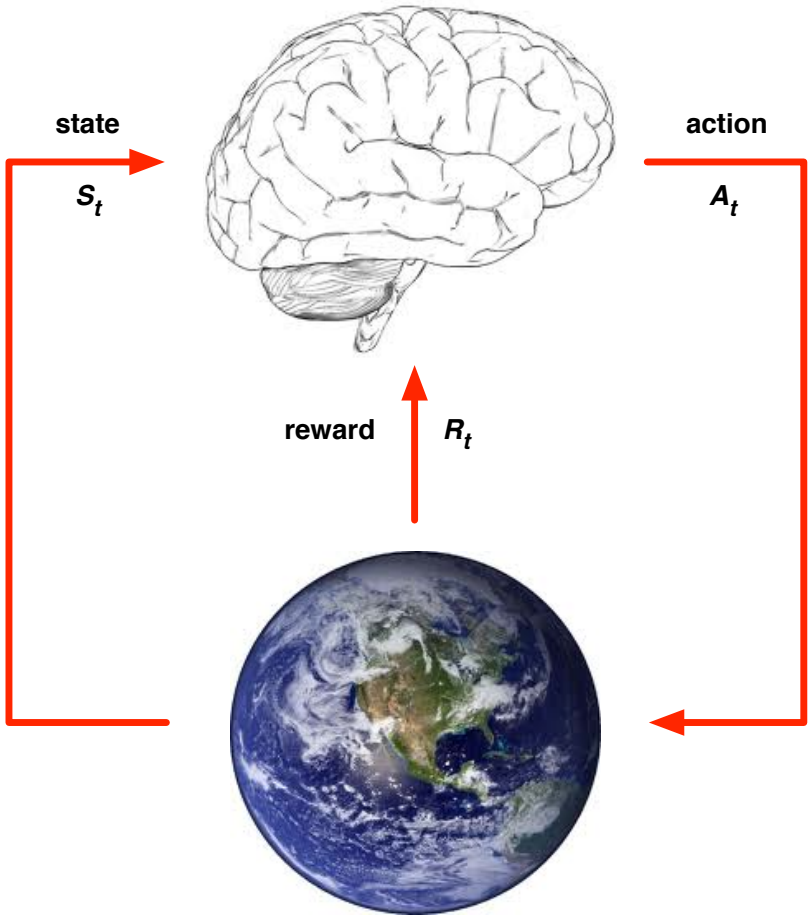
Many Faces of Reinforcement Learning



Branches of Machine Learning



Agent and Environment



- At each step t the agent:
  - Executes action A_t
  - Receives state S_t
  - Receives scalar reward R_t
- The environment:
  - Receives action A_t
  - Emits state S_{t+1}
  - Emits scalar reward R_{t+1}
- t increments at environment

Characteristics of Reinforcement Learning

What makes reinforcement learning different from other machine learning paradigms?

- There is no supervisor, only a *reward* signal
- Feedback is delayed, not instantaneous
- Time really matters (sequential, non i.i.d data)
- Agent's actions affect the subsequent data it receives

Examples of Reinforcement Learning

- Fly stunt manoeuvres in a helicopter
- Manage an investment portfolio
- Make a humanoid robot walk
- Play many different Atari games better than humans

Super Mario

DeepMind AI Just Taught Itself to Walk

Rewards

- A reward R_t is a scalar feedback signal
- Indicates how well agent is doing at step t
- The agent's job is to maximise cumulative reward

Reinforcement learning is based on the reward hypothesis

Definition (Reward Hypothesis)

All goals can be described by the maximisation of expected cumulative reward

Examples of Rewards

- Fly stunt manoeuvres in a helicopter
  - +ve reward for following desired trajectory
  - −ve reward for crashing
- Manage an investment portfolio
  - +ve reward for each $ in bank
- Make a humanoid robot walk
  - +ve reward for forward motion
  - −ve reward for falling over
- Play many different Atari games better than humans
  - +/−ve reward for increasing/decreasing score

Sequential Decision Making

- Goal: *select actions to maximise total future reward*
- Actions may have long term consequences
- Reward may be delayed
- It may be better to sacrifice immediate reward to gain more long-term reward
- Examples:
  - A financial investment (may take months to mature)
  - Refuelling a helicopter (might prevent a crash in several hours)

Outline

1 What is Reinforcement Learning?

2 Markov Decision Process

3 Value Based Methods
- Dynamic Programming
- Q-Learning
- Value Function Approximation

4 Policy Gradient Methods
- Likelihood Ratio Gradient
- Variance Reduction
- Actor Critic

5 Summary

Introduction to MDPs

- *Markov decision processes* formally describe an environment for reinforcement learning
- Where the environment is *fully observable*
- i.e. The current *state* completely characterises the process
- Almost all RL problems can be formalised as MDPs, e.g.
  - Optimal control primarily deals with continuous MDPs
  - Partially observable problems can be converted into MDPs
  - Bandits are MDPs with one state

Markov Property

"The future is independent of the past given the present"

Definition

A state S_t is Markov if and only if

$$P[S_{t+1}|S_t] = P[S_{t+1}|S_1, ..., S_t]$$

- The state captures all relevant information from the history
- Once the state is known, the history can be thrown away
- The state is a sufficient statistic of the future

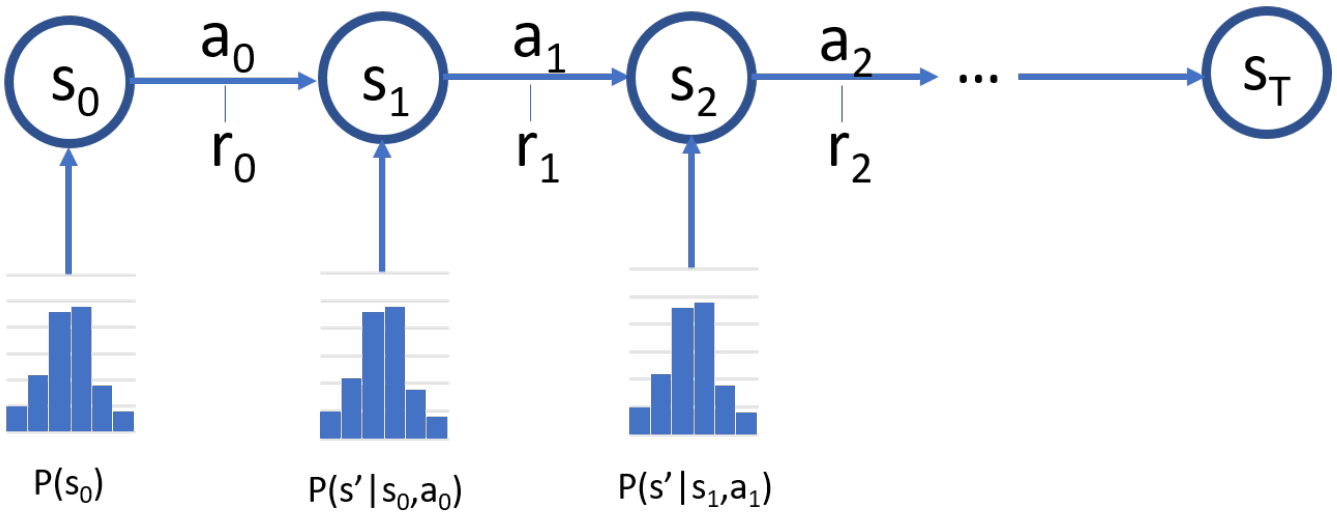
Markov Decision Process

Definition

A *Markov Decision Process* is a tuple $< \mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R}, \gamma >$

- $\mathcal{S}$ is a finite set of states
- $\mathcal{A}$ is a finite set of actions
- $\mathcal{P}$ is a state transition probability matrix, $\mathcal{P}^{a}_{ss'} = P[S_{t+1} = s' | S_t = s, A_t = a]$
- $\mathcal{R}$ is a reward function, $r_t = \mathcal{R}(s_t, a_t)$
- γ is a discount factor, $\gamma \in [0, 1]$

Markov Decision Process



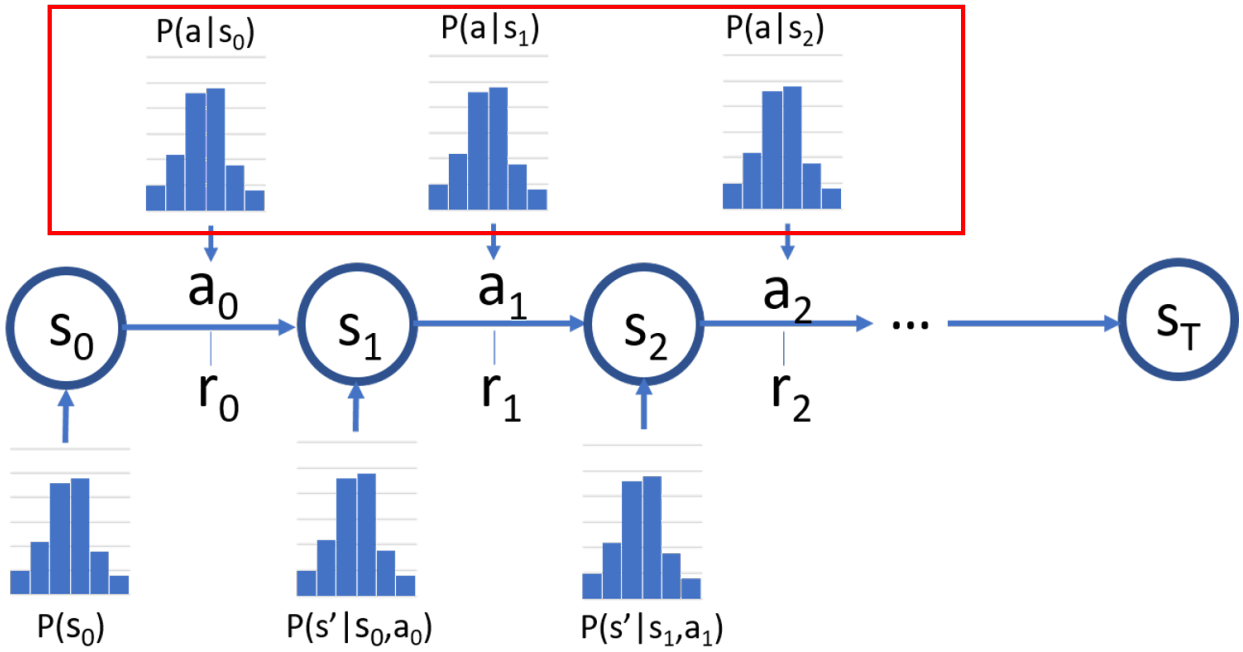
- At time step $t = 0$, environment samples initial state $s_0 \sim P(s_0)$
- Then,agent selects action a_t
- Environment emits reward $r_t = \mathcal{R}(s_t, a_t)$ and samples next state $s_{t+1} \sim P[s^{'}|s_t, a_t]$

Policy

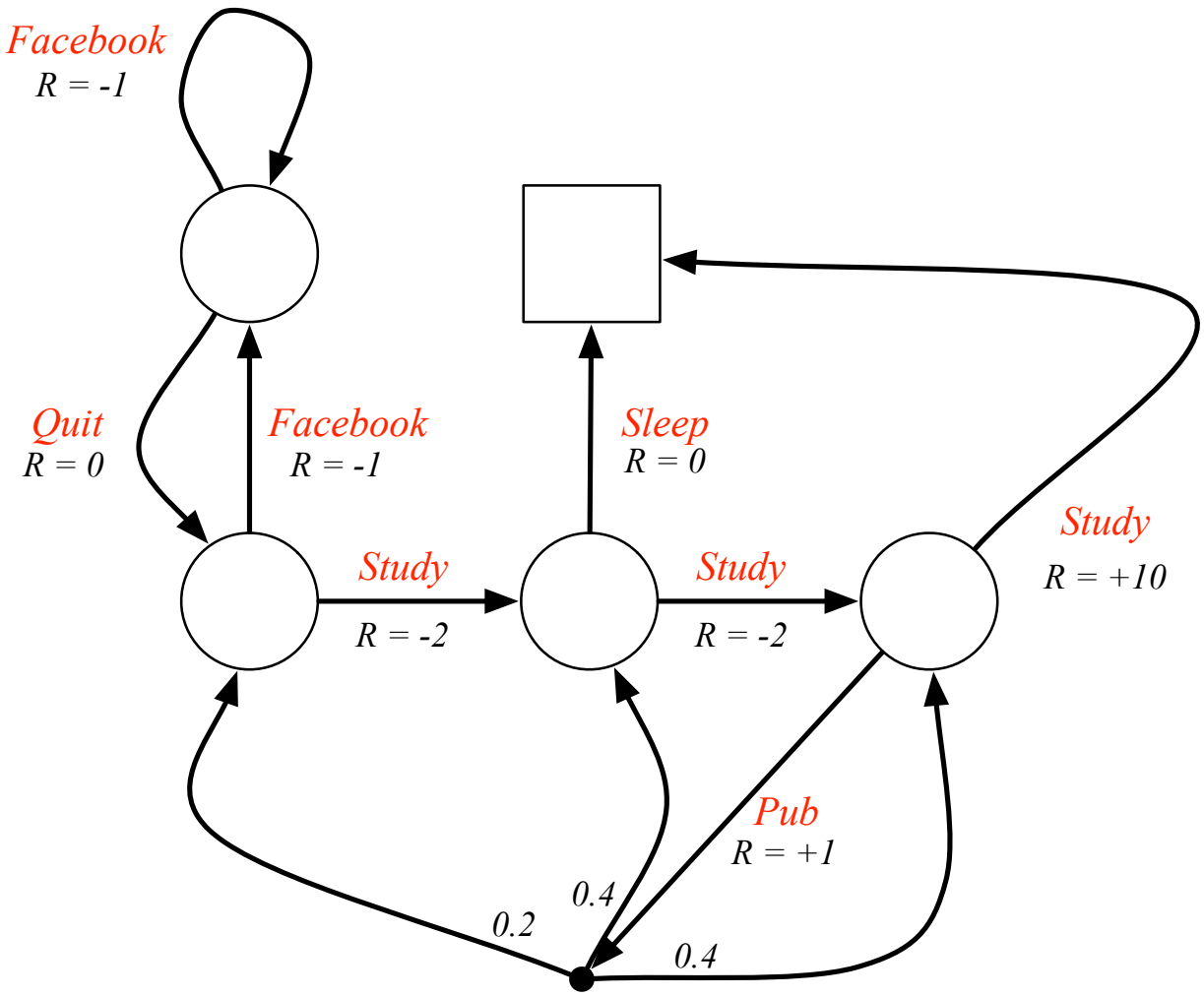
Policy

A policy defines the behaviour of an agent

$$\pi(a|s) = P[A_t = a|S_t = s]$$



Example: Student MDP



Return

Definition

The *return* G_t is the total discounted reward from time-step t

$$G_t = R_{t+1} + \gamma R_{t+2} + ... = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}$$

- The discount $\gamma \in [0, 1]$ is the present value of future rewards
- The value of receiving reward R after $k + 1$ time-steps is $\gamma^k R$
- This values immediate reward above delayed reward
  - γ close to 0 leads to "myopic" evaluation
  - γ close to 1 leads to "far-sighted" evaluation

Why discount?

Most Markov reward and decision processes are discounted. Why?

- Mathematically convenient to discount rewards
- Avoids infinite returns in cyclic Markov processes
- Uncertainty about the future may not be fully represented
- If the reward is financial, immediate rewards may earn more interest than delayed rewards
- Animal/human behaviour shows preference for immediate reward
- It is sometimes possible to use undiscounted Markov reward processes (i.e. $\gamma = 1$), e.g. if all sequences terminate.

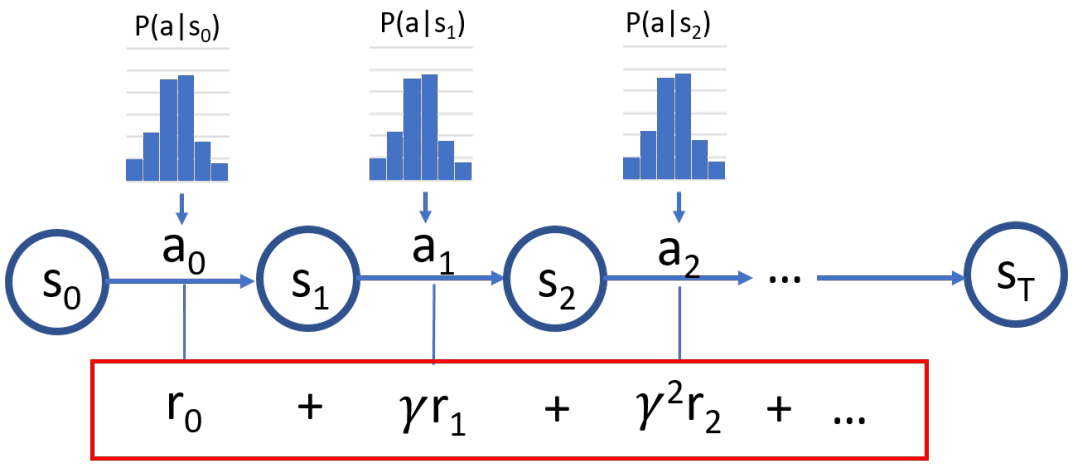
Objective

Objective

Find the optimal policy π^* that maximizes the expected cumulative discounted reward:

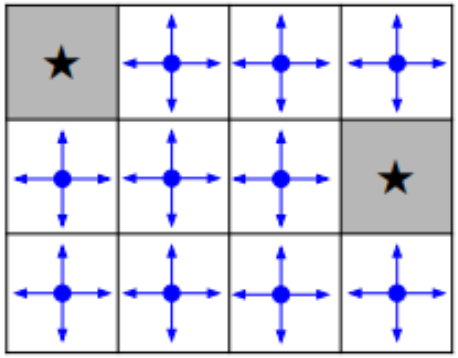
$$\pi^* = \arg\max_{\pi} \mathbb{E}\left[\sum_{t \geq 0} \gamma^t r_t | \pi\right]$$

Here γ encourages agent to get reward as fast as possible

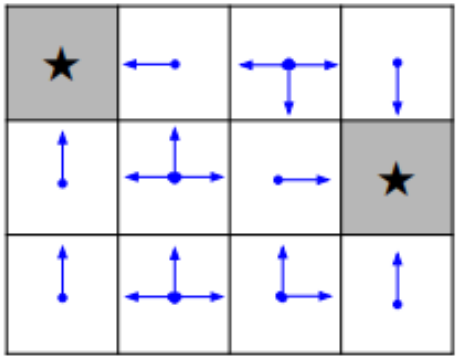


Grid World

- $S = \{grids\}$
- $A = \{right, left, up, down\}$
- Reward: $r = -1$ for each transition
- Objective: reach the terminate states by least steps



Random Policy



Optimal Policy

State-Value Function

Definition

The state-value function $V_\pi(s)$ is the expected cumulative reward achieved by following the policy π from state s:

$$V_\pi(s) = \mathbb{E}\left[\sum_{i \geq t} \gamma^{i-t} r_i | s_t = s, \pi\right]$$

- $V_\pi(s)$ evaluates how good the state s is by following π
- Optimal state-value function $V_*(s)$ evaluates how good the state s is by following optimal policy π_*

Action-Value Function

Definition

The action-value function $Q_\pi(s, a)$ is the expected cumulative reward achieved by taking action a in state s and then following the policy π:

$$Q_\pi(s, a) = \mathbb{E}\left[\sum_{i \geq t} \gamma^{i-t} r_i | s_t = s, a_t = a, \pi\right]$$

- $Q_\pi(s, a)$ evaluates how good it is to take action a in state s and then follow policy π
- Optimal action-value function $Q_*(s, a)$ evaluates how good it is to take action a in state s and then follow optimal policy π_*

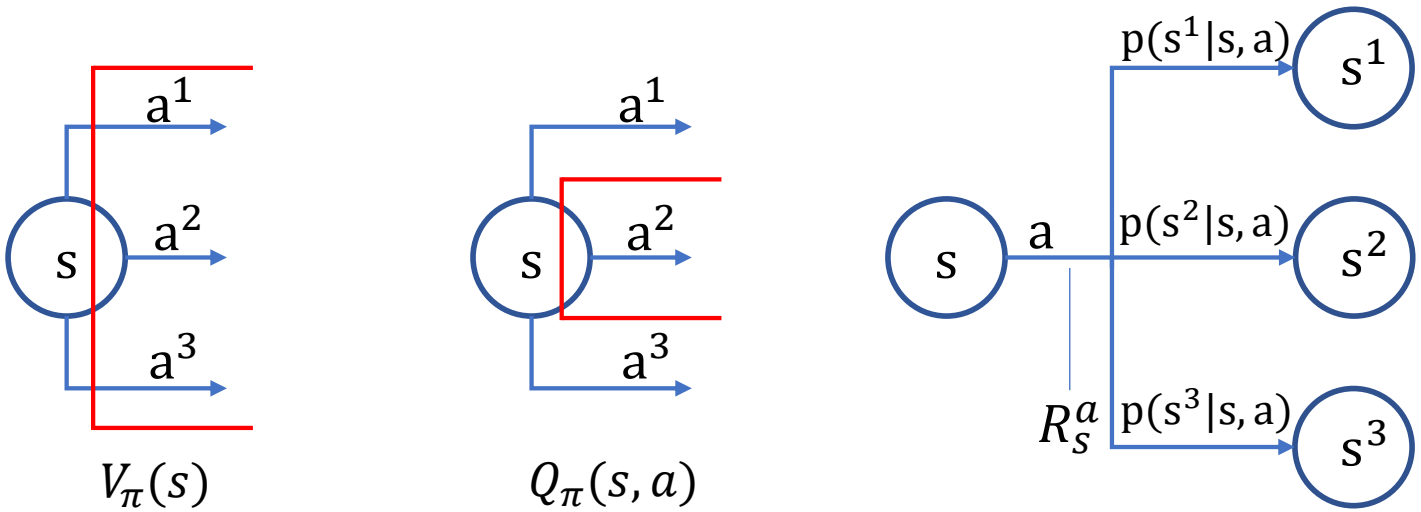
$V_\pi(s)$ and $Q_\pi(s,a)$

- Relation between $V_\pi(s)$ and $Q_\pi(s,a)$:

$$V_\pi(s) = \sum_a \pi(a|s) Q_\pi(s,a)$$

$$Q_\pi(s,a) = R_s^a + \gamma \sum_{s' \in \mathcal{S}} P_{ss'}^a V_\pi(s')$$

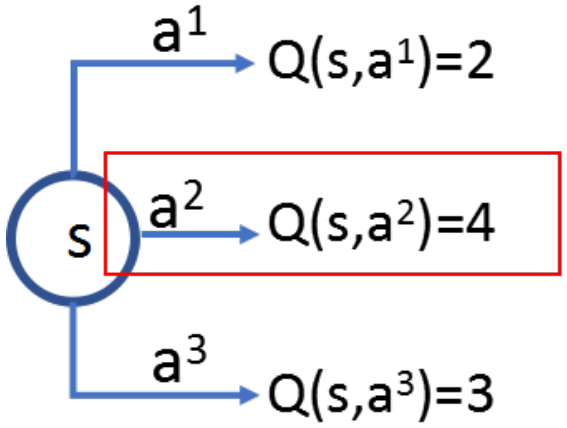
Here s' denotes next state.



Optimal Policy

If we know the optimal action-value function, then we derive the optimal policy:

$$\pi_*(a|s) = \begin{cases} 1 & a = \arg\max_a Q_*(s,a) \\ 0 & others \end{cases}$$



And we have:

$$V_*(s) = \max_a Q_*(s,a)$$

Bellman Equation

The state-value function can be decomposed into *immediate reward* R_{t+1} and *discounted value of successor state* $\gamma v_\pi(S_{t+1})$,

$$\begin{aligned} v_\pi(s) &= \mathbb{E}_\pi[G_t \mid S_t = s] \\ &= \mathbb{E}_\pi[R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \ldots \mid S_t = s] \\ &= \mathbb{E}_\pi[R_{t+1} + \gamma(R_{t+2} + \gamma R_{t+3} + \ldots) \mid S_t = s] \\ &= \mathbb{E}_\pi[R_{t+1} + \gamma G_{t+1} \mid S_t = s] \\ &= \mathbb{E}_\pi[R_{t+1} + \gamma v_\pi(S_{t+1}) \mid S_t = s] \end{aligned}$$

The action-value function can similarly be decomposed,

$$q_\pi(s, a) = \mathbb{E}_\pi[R_{t+1} + \gamma q_\pi(S_{t+1}, A_{t+1}) \mid S_t = s, A_t = a]$$

Bellman Equation in Matrix Form

The Bellman equation can be expressed concisely using matrices,

$$v = \mathcal{R} + \gamma \mathcal{P} v$$

where v is a column vector with one entry per state

$$\begin{bmatrix} v(1) \\ \vdots \\ v(n) \end{bmatrix} = \begin{bmatrix} \mathcal{R}_1 \\ \vdots \\ \mathcal{R}_n \end{bmatrix} + \gamma \begin{bmatrix} \mathcal{P}_{11} & \dots & \mathcal{P}_{1n} \\ \vdots & & \\ \mathcal{P}_{11} & \dots & \mathcal{P}_{nn} \end{bmatrix} \begin{bmatrix} v(1) \\ \vdots \\ v(n) \end{bmatrix}$$

Outline

1 What is Reinforcement Learning?

2 Markov Decision Process

3 Value Based Methods
- Dynamic Programming
- Q-Learning
- Value Function Approximation

4 Policy Gradient Methods
- Likelihood Ratio Gradient
- Variance Reduction
- Actor Critic

5 Summary

What is Dynamic Programming?

Dynamic sequential or temporal component to the problem
Programming optimising a "program", i.e. a policy

- c.f. linear programming

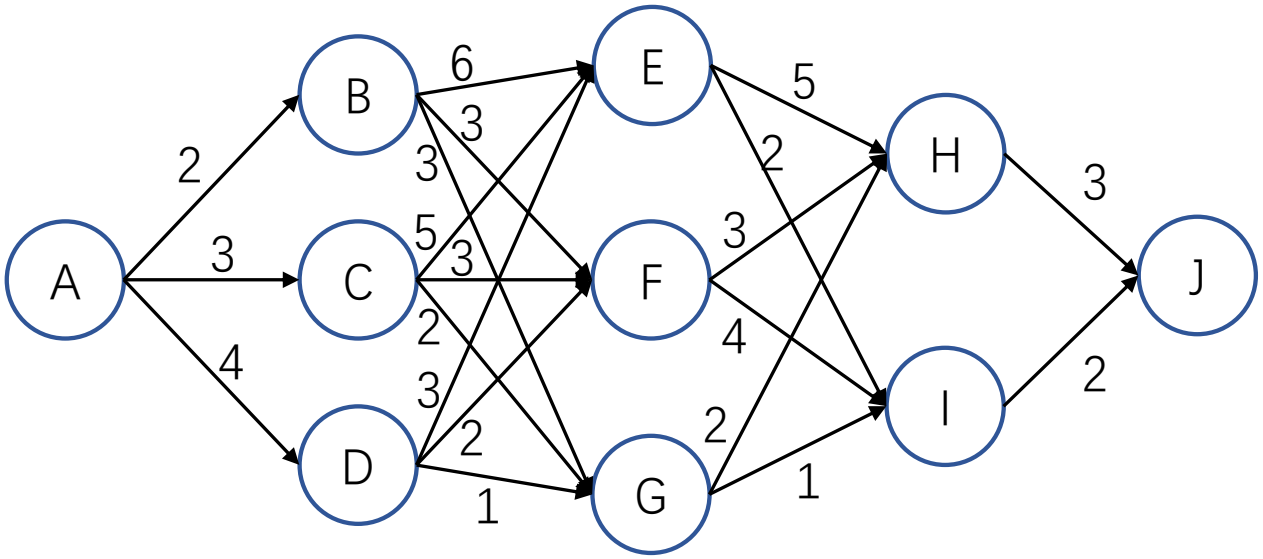
- A method for solving complex problems
- By breaking them down into subproblems
  - Solve the subproblems
  - Combine solutions to subproblems

Other Applications of Dynamic Programming

Dynamic programming is used to solve many other problems, e.g.

- Scheduling algorithms
- String algorithms (e.g. sequence alignment)
- Graph algorithms (e.g. shortest path algorithms)
- Graphical models (e.g. Viterbi algorithm)

Shortest Path



$$distance(A, B) + shortest(B, J)$$

$$distance(A, C) + shortest(C, J)$$

$$distance(A, D) + shortest(D, J)$$

Requirements for Dynamic Programming

Dynamic Programming is a very general solution method for problems which have two properties:

- Optimal substructure
  - *Principle of optimality* applies
  - Optimal solution can be decomposed into subproblems
- Overlapping subproblems
  - Subproblems recur many times
  - Solutions can be cached and reused
- Markov decision processes satisfy both properties
  - Bellman equation gives recursive decomposition
  - Value function stores and reuses solutions

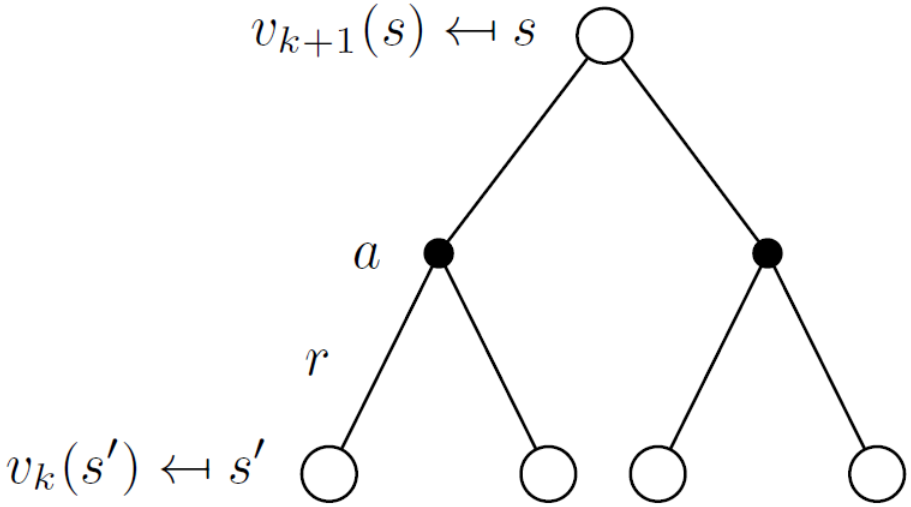
Planning by Dynamic Programming

- Dynamic programming assumes full knowledge of the MDP
- It is used for planning in an MDP
- For prediction:
  - Input: MDP $< \mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R}, \gamma >$ and policy π
  - Output: value function v_π
- Or for control:
  - Input: MDP $< \mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R}, \gamma >$
  - Output: optimal value function v_* and optimal policy π_*

Iterative Policy Evaluation

- Problem: evaluate a given policy π
- Solution: iterative application of Bellman expectation backup
- $v_1 \rightarrow v_2 \rightarrow \ldots \rightarrow v_\pi$
- Using synchronous backups,
  - At each iteration $k+1$
  - For all states $s \in \mathcal{S}$
  - Update $v_{k+1}(s)$ from $v_k(s')$
  - where s' is a successor state of s
- Convergence to v_π will be proven later

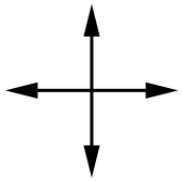
Iterative Policy Evaluation (2)



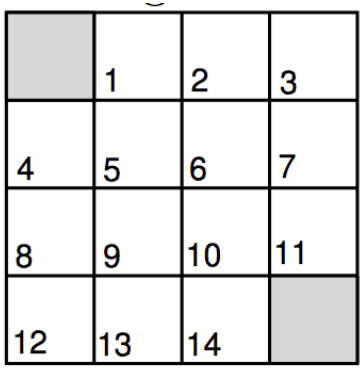
$$v_{k+1}(s) = \sum_{a \in \mathcal{A}} \pi(a|s) \left(\mathcal{R}_s^a + \gamma \sum_{s' \in \mathcal{S}} \mathcal{P}_{ss'}^a v_k(s') \right)$$

$$\mathbf{v}^{k+1} = \boldsymbol{\mathcal{R}}^{\boldsymbol{\pi}} + \gamma \boldsymbol{\mathcal{P}}^{\boldsymbol{\pi}} \mathbf{v}^k$$

Evaluating a Random Policy in the Small Gridworld



actions



$r = -1$
on all transitions

- Undiscounted episodic MDP $\gamma = 1$
- Nonterminal states 1,...,14
- One terminal state (shown twice as shaded squares)
- Actions leading out of the grid leave state unchanged
- Reward is -1 until the terminal state is reached
- Agent follows uniform random policy

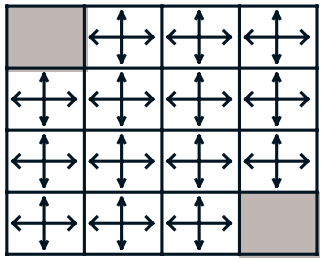
$$\pi(n|\cdot) = \pi(e|\cdot) = \pi(s|\cdot) = \pi(w|\cdot) = 0.25$$

Iterative Policy Evaluation in Small Gridworld

	v_k for the Random Policy	Greedy Policy w.r.t. v_k

$k = 0$

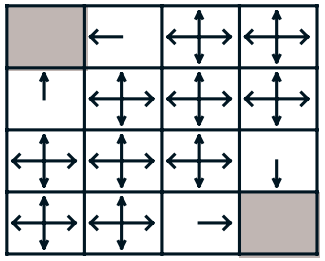
0.0	0.0	0.0	0.0
0.0	0.0	0.0	0.0
0.0	0.0	0.0	0.0
0.0	0.0	0.0	0.0



random policy

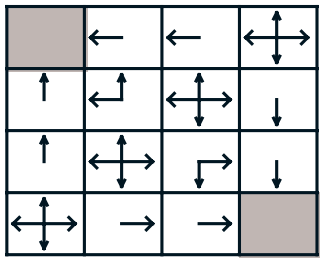
$k = 1$

0.0	-1.0	-1.0	-1.0
-1.0	-1.0	-1.0	-1.0
-1.0	-1.0	-1.0	-1.0
-1.0	-1.0	-1.0	0.0



$k = 2$

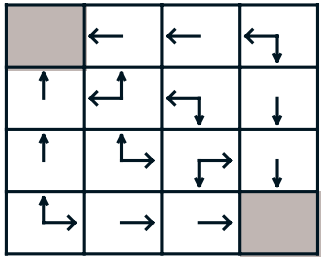
0.0	-1.7	-2.0	-2.0
-1.7	-2.0	-2.0	-2.0
-2.0	-2.0	-2.0	-1.7
-2.0	-2.0	-1.7	0.0



Iterative Policy Evaluation in Small Gridworld (2)

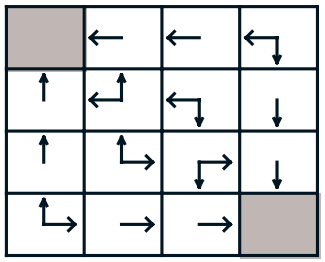
$k = 3$

0.0	-2.4	-2.9	-3.0
-2.4	-2.9	-3.0	-2.9
-2.9	-3.0	-2.9	-2.4
-3.0	-2.9	-2.4	0.0



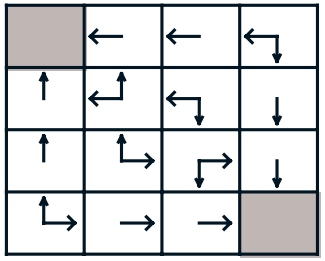
$k = 10$

0.0	-6.1	-8.4	-9.0
-6.1	-7.7	-8.4	-8.4
-8.4	-8.4	-7.7	-6.1
-9.0	-8.4	-6.1	0.0



$k = \infty$

0.0	-14.	-20.	-22.
-14.	-18.	-20.	-20.
-20.	-20.	-18.	-14.
-22.	-20.	-14.	0.0



optimal policy

How to Improve a Policy

- Given a policy π
  - Evaluate the policy π

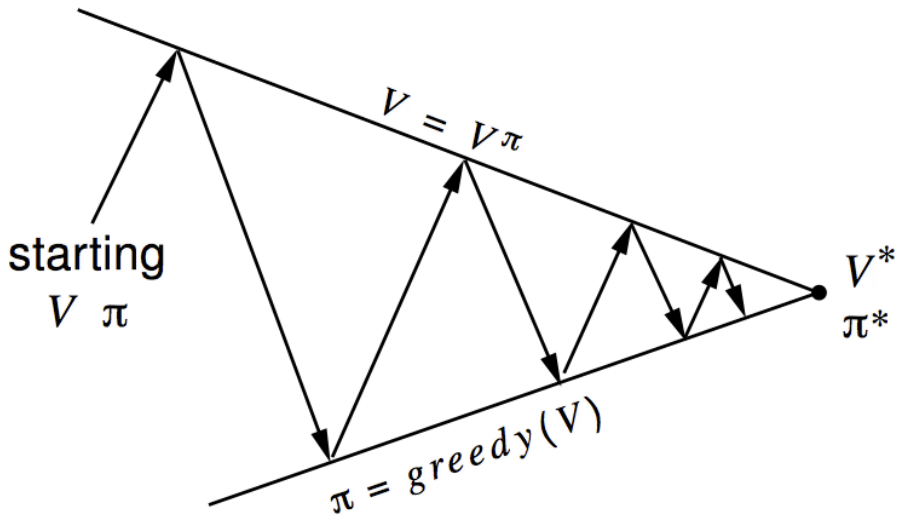
$$v_\pi(s) = \mathbb{E}[R_{t+1} + \gamma R_{t+2} + ... \mid S_t = s]$$

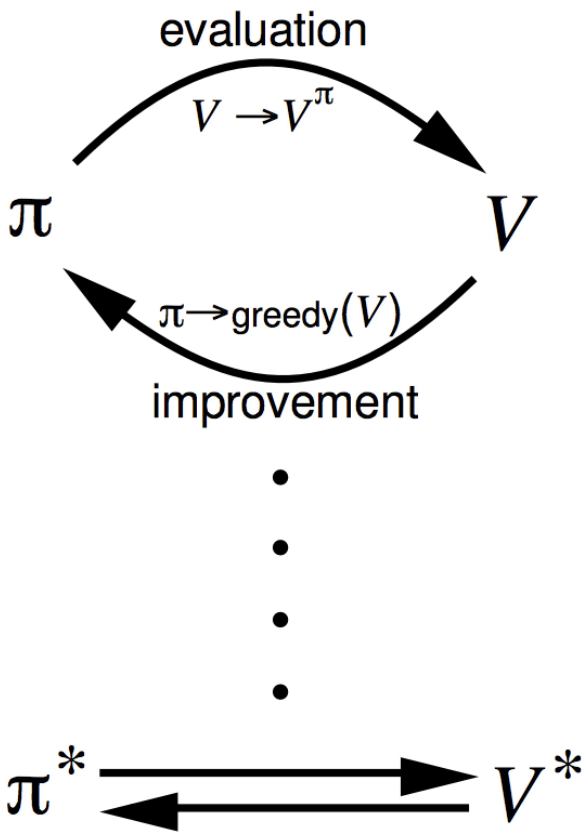
  - Improve the policy by acting greedily with respect to v_π

$$\pi' = greedy(v_\pi)$$

- In Small Gridworld improved policy was optimal, $\pi' = \pi^*$
- In general, need more iterations of improvement / evaluation
- But this process of policy iteration always converges to π^*

Policy Iteration





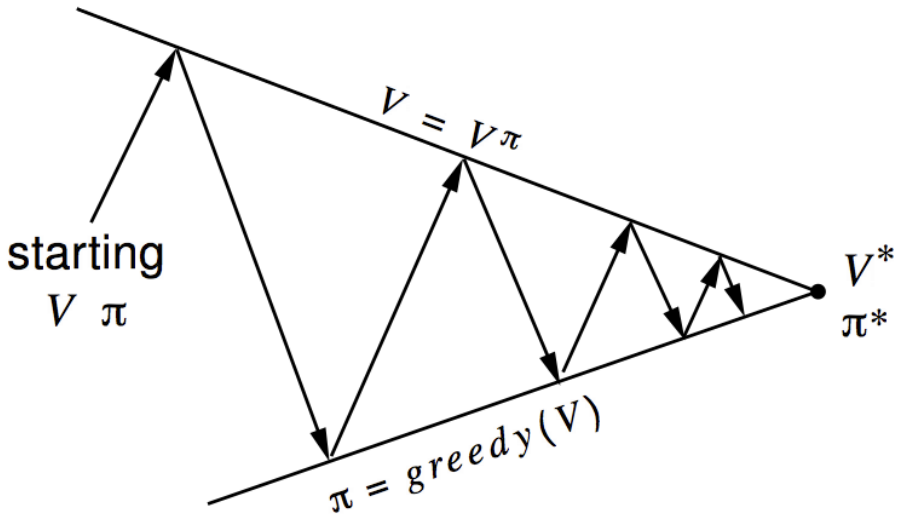
Policy evaluation Estimate v_π

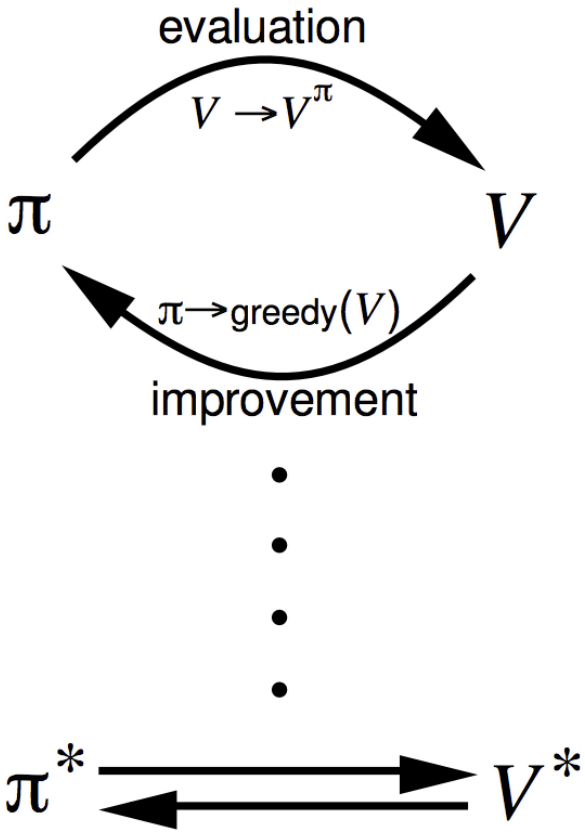
Iterative policy evaluation

Policy improvement Generate $\pi' \geq \pi$

Greedy policy improvement

Generalised Policy Iteration





Policy evaluation Estimate v_π

Any policy evaluation algorithm

Policy improvement Generate $\pi' \geq \pi$

Any policy improvement algorithm

Deterministic Value Iteration

- If we know the solution to subproblems $v_*(s')$
- Then solution $v_*(s)$ can be found by one-step lookahead

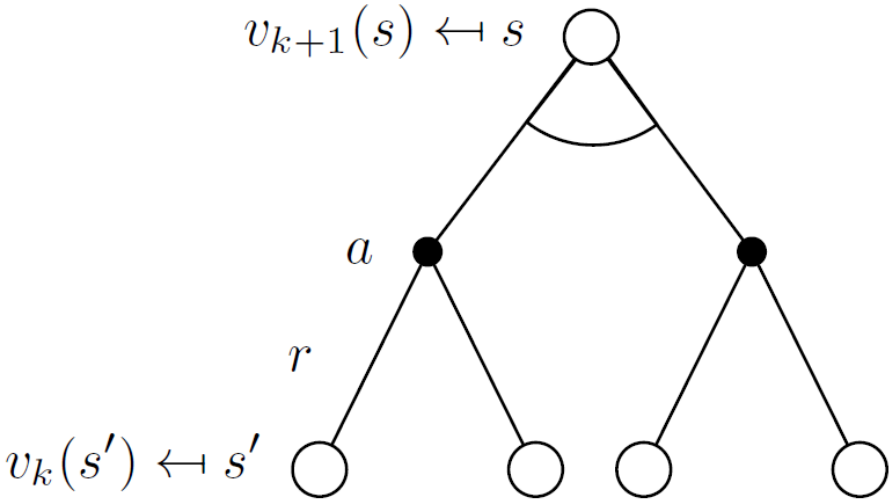
$$v_*(s) \leftarrow \max_{a \in \mathcal{A}} \mathcal{R}_s^a + \gamma \sum_{s' \in \mathcal{S}} \mathcal{P}_{ss'}^a v_*(s')$$

- The idea of value iteration is to apply these updates iteratively
- Intuition: start with final rewards and work backwards
- Still works with loopy, stochastic MDPs

Value Iteration

- Problem: find optimal policy π
- Solution: iterative application of Bellman optimality backup
- $v_1 \rightarrow v_2 \rightarrow \ldots \rightarrow v_*$
- Using synchronous backups,
  - At each iteration $k+1$
  - For all states $s \in \mathcal{S}$
  - Update $v_{k+1}(s)$ from $v_k(s')$
- Convergence to v_* will be proven later
- Unlike policy iteration, there is no explicit policy
- Intermediate value functions may not correspond to any policy

Value Iteration (2)



$$v_{k+1}(s) = \max_{a \in \mathcal{A}} \left(\mathcal{R}_s^a + \gamma \sum_{s' \in \mathcal{S}} \mathcal{P}_{ss'}^a v_k(s') \right)$$

$$\mathbf{v}_{k+1} = \max_{a \in \mathcal{A}} \boldsymbol{\mathcal{R}}^a + \gamma \boldsymbol{\mathcal{P}}^a \mathbf{v}_k$$

Value Function ∞-Norm

- We will measure distance between state-value functions u and v by the $\infty-$norm
- i.e. the largest difference between state values,

$$||u - v||_\infty = \max_{s \in \mathcal{S}} |u(s) - v(s)|$$

Bellman Expectation Backup is a Contraction

- Define the *Bellman expectation backup operator* T^π

$$T^\pi(v) = \mathcal{R}^\pi + \gamma \mathcal{P}^\pi v$$

- This operator is a $\gamma-$contraction, i.e. it makes value functions closer by at least γ,

$$\begin{aligned} ||T^\pi(u) - T^\pi(v)||_\infty &= ||(\mathcal{R}^\pi + \gamma \mathcal{P}^\pi u) - (\mathcal{R}^\pi + \gamma \mathcal{P}^\pi v)||_\infty \\ &= ||\gamma \mathcal{P}^\pi (u - v)||_\infty \\ &\leq ||\gamma \mathcal{P}^\pi ||u - v||_\infty||_\infty \\ &\leq \gamma ||u - v||_\infty \end{aligned}$$

Contraction Mapping Theorem

Theorem (Contraction Mapping Theorem)

For any metric space $\mathcal{V}$ that is complete (i.e. closed) under an operator $T(v)$, where T is a $\gamma-$contraction,

- T converges to a unique fixed point
- At a linear convergence rate of γ

Convergence of Policy Iteration and Value Iteration

Convergence of Policy Iteration

- The Bellman expectation operator T^π has a unique fixed point
- Iterative policy evaluation converges on v_π based on contraction mapping theorem
- Policy iteration converges on π_* based on policy improvement

Convergence of Value Iteration

- The Bellman optimality operator T^* has a unique fixed point
- Value iteration converges on v_* based on contraction mapping theorem

Model-Free Reinforcement Learning

- Dynamic Programming:
  - Prediction or control
  - Solve a *known* MDP
- Model-Free Methods:
  - Prediction: Estimate the value function of an *unknown* MDP
  - Control: Optimise the value function of an *unknown* MDP

Monte-Carlo Reinforcement Learning

- MC methods learn directly from episodes of experience
- MC is *model-free*: no knowledge of MDP transitions
- MC learns from *complete* episodes: no bootstrapping
- MC uses the simplest possible idea: value = mean return
- Caveat: can only apply MC to episodic MDPs
  - All episodes must terminate

Monte-Carlo Policy Evaluation

- Goal: learn v_π from episodes of experience under policy π

$$S_1, A_1, R_2, ..., S_k \sim \pi$$

- Recall that the *return* is the total discounted reward:

$$G_t = R_{t+1} + \gamma R_{t+2} + ... + \gamma^{T-1} R_T$$

- Recall that the value function is the expected return:

$$v_\pi(s) = \mathbb{E}_\pi[G_t | S_t = s]$$

- Monte-Carlo policy evaluation uses *empirical mean* return instead of *expected* return

Monte-Carlo Policy Evaluation

- To evaluate state s
- Every time-step t that state s is visited in an episode
- Increment counter $N(s) \leftarrow N(s) + 1$
- Increment total return $S(s) \leftarrow S(s) + G_t$
- Value is estimated by mean return $V(s) = S(s)/N(s)$
- By law of large numbers, $V(s) \rightarrow v_\pi(s)$ as $N(s) \rightarrow \infty$

Temporal-Difference Learning

- TD methods learn directly from episodes of experience
- TD is *model-free*: no knowledge of MDP transitions
- TD learns from *incomplete* episodes, by *bootstrapping*
- TD updates a guess towards a guess

MC and TD

- Goal: learn v_π online from experience under policy π
- Incremental every-visit Monte-Carlo
  - Update value $V(S_t)$ toward actual return G_t

$$V(S_t) \leftarrow V(S_t) + \alpha(G_t - V(S(t)))$$

- Simplest temporal-difference learning algorithm: TD(0)
  - Update value $V(S_t)$ toward estimated return $R_{t+1} + \gamma V(S_{t+1})$

$$V(S_t) \leftarrow V(S_t) + \alpha(R_{t+1} + \gamma V(S_{t+1)} - V(S(t))$$

  - $R_{t+1} + \gamma V(S_{t+1})$ is called the *TD target*
  - $\delta_t = R_{t+1} + \gamma V(S_{t+1}) - V(S_t)$ is called the *TD error*

Advantages and Disadvantages of MC vs. TD

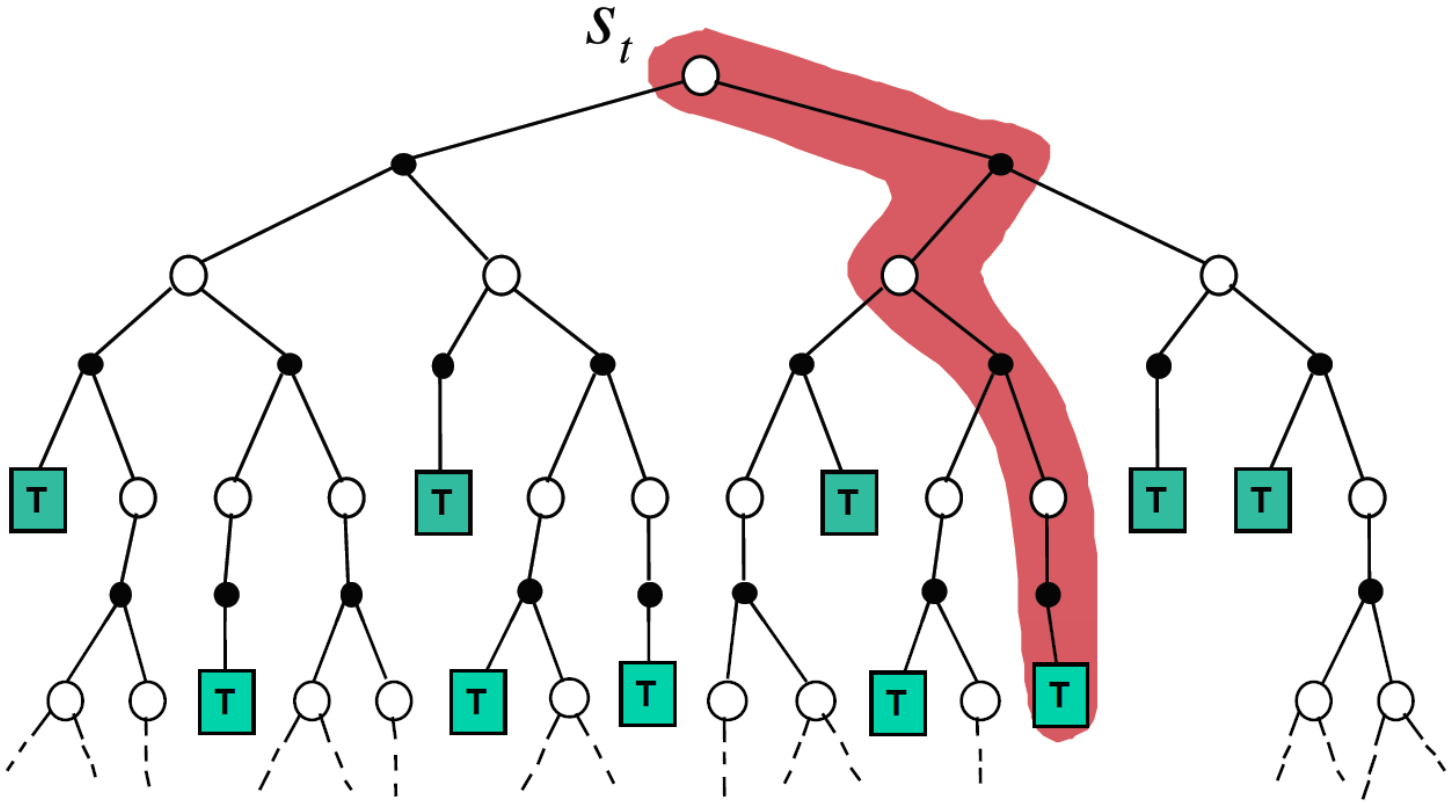
- TD can learn before knowing the final outcome
  - TD can learn online after every step
  - MC must wait until end of episode before return is known
- TD can learn without the final outcome
  - TD can learn from incomplete sequences
  - MC can only learn from complete sequences
  - TD works in continuing (non-terminating) environments
  - MC only works for episodic (terminating) environments

Advantages and Disadvantages of MC vs. TD (2)

- MC has high variance, zero bias
  - Good convergence properties
  - (even with function approximation)
  - Not very sensitive to initial value
  - Very simple to understand and use
- TD has low variance, some bias
  - Usually more efficient than MC
  - TD(0) converges to $v_\pi(s)$
  - (but not always with function approximation)
  - More sensitive to initial value

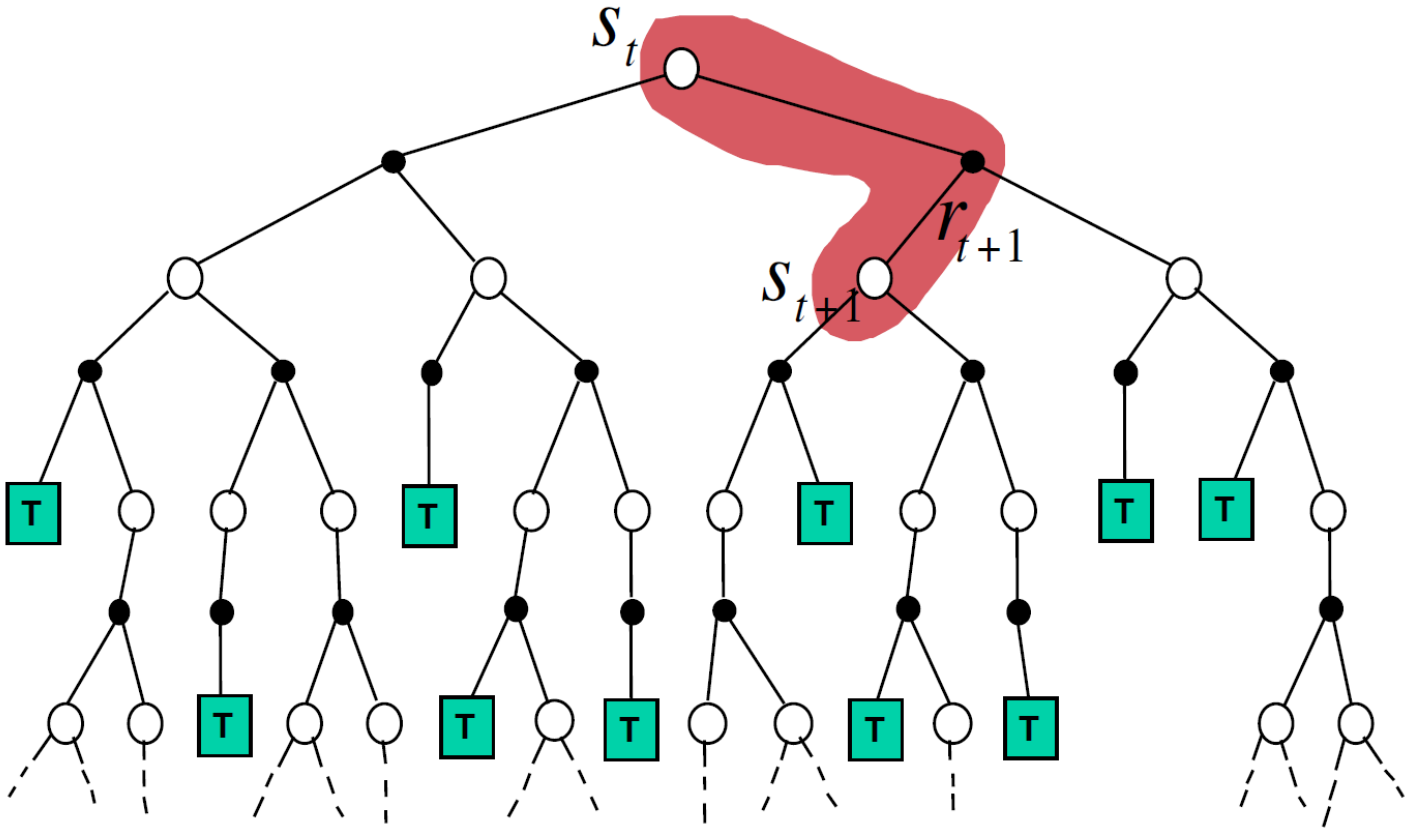
Monte-Carlo Backup

$$V(S_t) \leftarrow V(S_t) + \alpha\,(G_t - V(S_t))$$



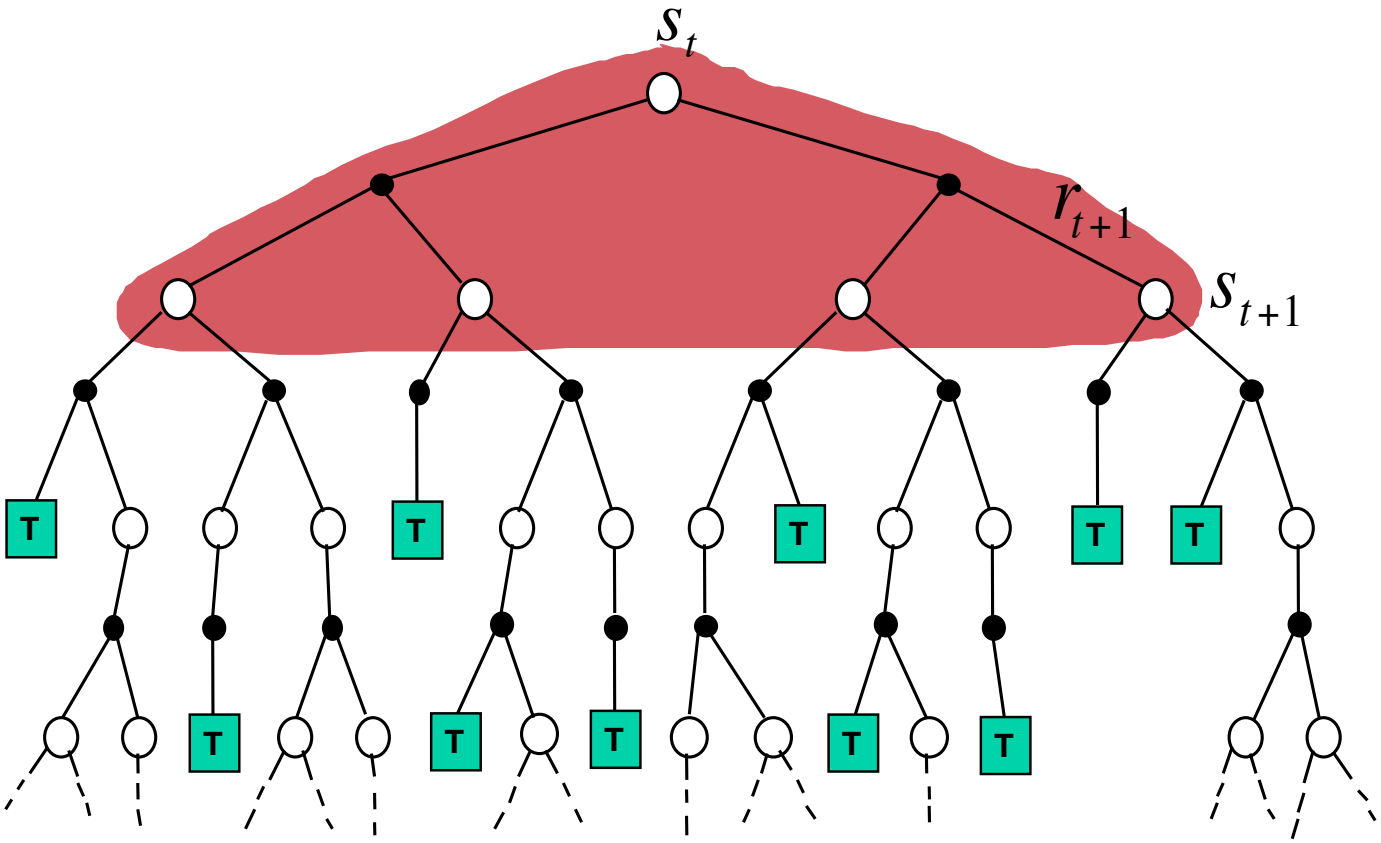
Temporal-Difference Backup

$$V(S_t) \leftarrow V(S_t) + \alpha\left(R_{t+1} + \gamma V(S_{t+1}) - V(S_t)\right)$$



Dynamic Programming Backup

$$V(S_t) \leftarrow \mathbb{E}_\pi \left[R_{t+1} + \gamma V(S_{t+1}) \right]$$



Model-Free Policy Iteration Using Action-Value Function

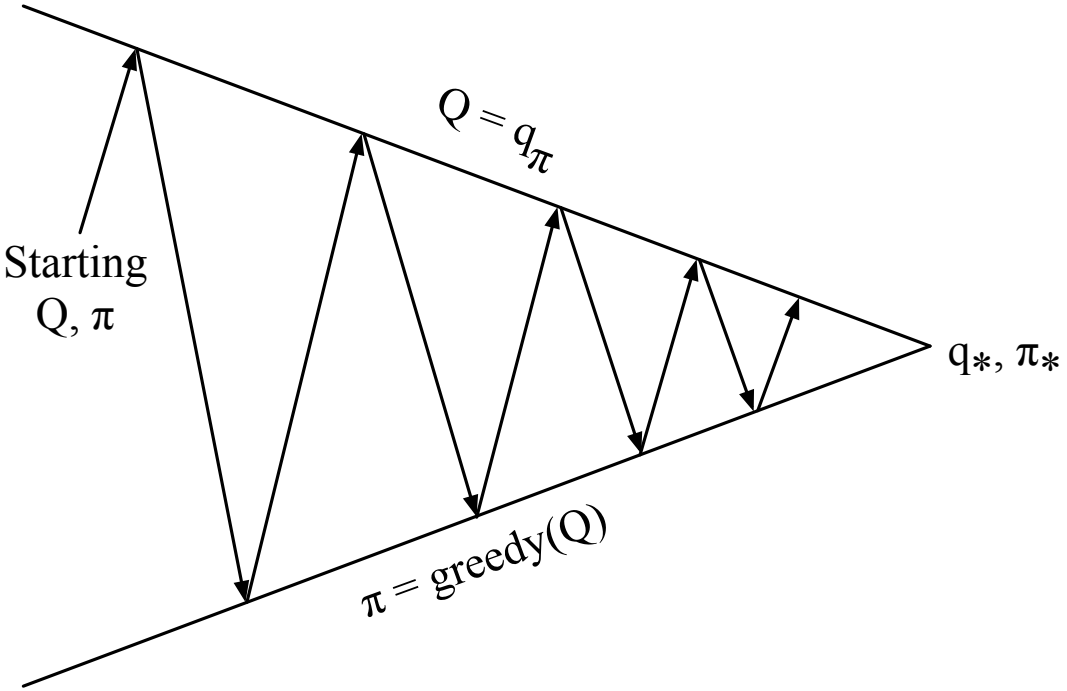
- Greedy policy improvement over $V(s)$ *requires model of MDP*

$$\pi'(s) = \arg\max_{a \in \mathcal{A}} \mathcal{R}_s^a + \mathcal{P}_{ss'}^a V(s')$$

- Greedy policy improvement over $Q(s, a)$ is *model-free*

$$\pi'(s) = \arg\max_{a \in \mathcal{A}} Q(s, a)$$

Generalised Policy Iteration with Action-Value Function



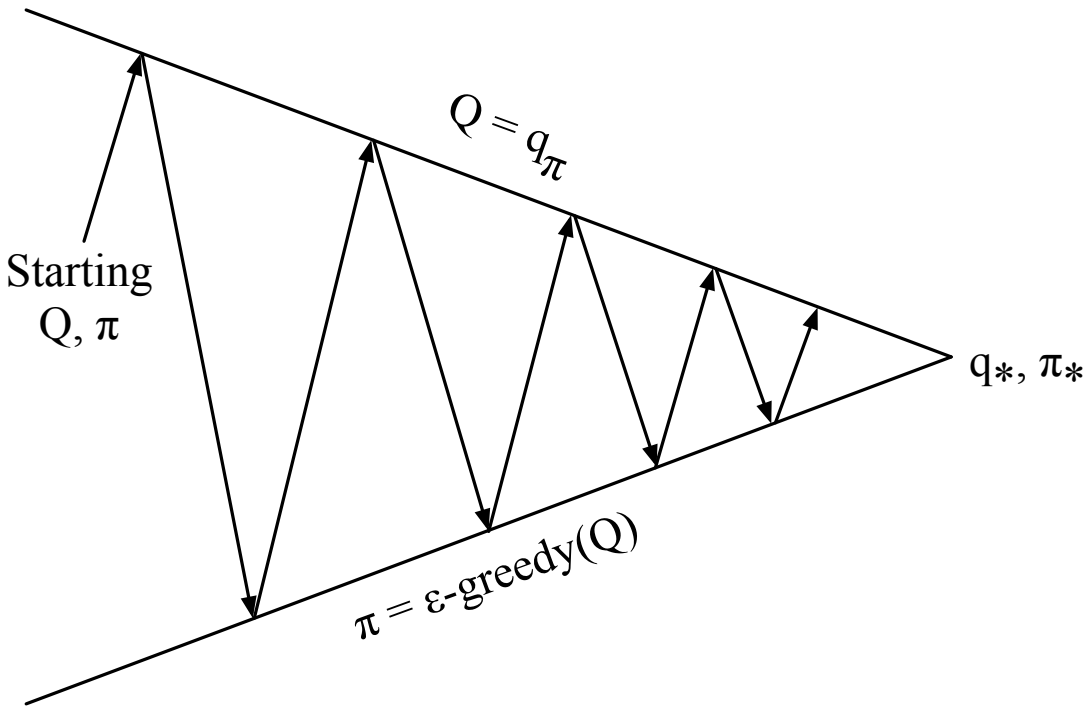
Policy evaluation Monte-Carlo or TD policy evaluation, $Q = q_\pi$
Policy improvement Greedy policy improvement?

ϵ-Greedy Exploration

- Simplest idea for ensuring continual exploration
- All m actions are tried with non-zero probability
- With probability $1 - \epsilon$ choose the greedy action
- With probability ϵ choose an action at random

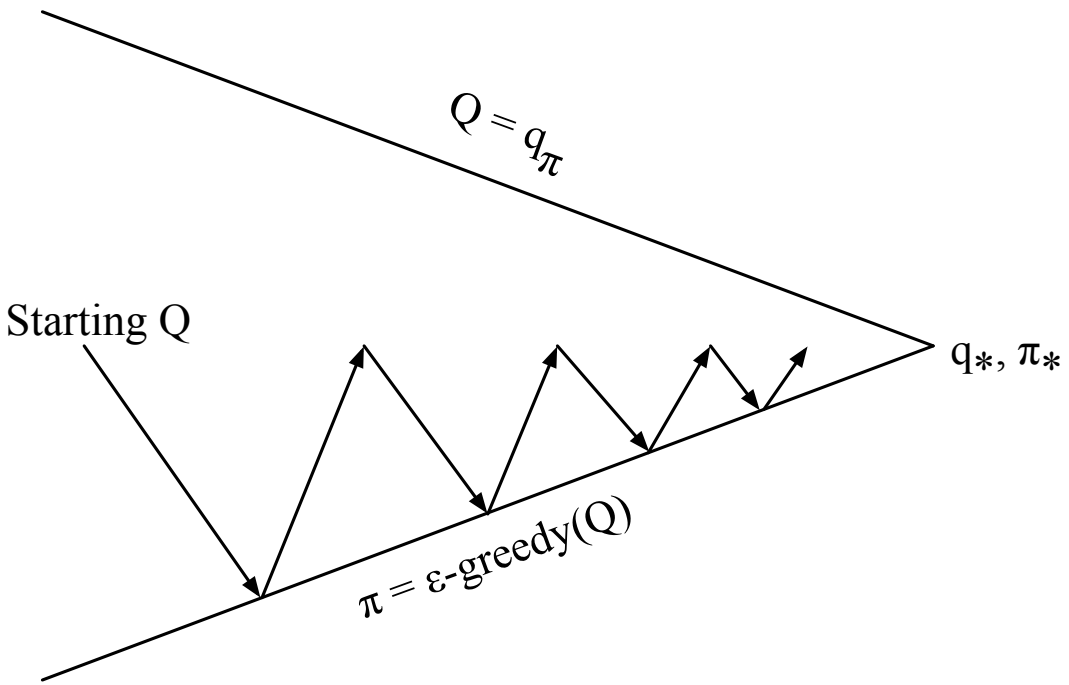
$$\pi(a|s) = \begin{cases} \epsilon/m + 1 - \epsilon & \text{if } a^* = \arg\max_{a \in \mathcal{A}} Q(s,a) \\ \epsilon/m & \text{otherwise} \end{cases}$$

Model-Free Policy Iteration



Policy evaluation Monte-Carlo or TD policy evaluation, $Q = q_\pi$
Policy improvement ϵ-greedy policy improvement

Model-Free Control



Every episode:

Policy evaluation Monte-Carlo or TD policy evaluation, $Q \approx q_\pi$

Policy improvement ϵ-greedy policy improvement

Q-Learning Algorithm

Initialize $Q(s, a), \forall s \in \mathcal{S}, a \in \mathcal{A}(s)$, arbitrarily, and $Q(terminal, \cdot) = 0$
Repeat (for each episode):
  Initialize S
  Repeat (for each step of episode):
    Choose A from S using policy derived from Q (e.g.,ϵ-greedy)
    Take action A, observe $\mathcal{R}$, $\mathcal{S}'$
    $Q(S, A) \leftarrow Q(S, A) + \alpha[\mathcal{R} + \gamma \max_a Q(S', a) - Q(S, A)]$
    $S \leftarrow S'$;
  Until S is terminal

Large-Scale Reinforcement Learning

Reinforcement learning can be used to solve *large* problems, e.g.

- Computer Go: 10^{170} states
- Helicopter: continuous state space

How can we scale up the model-free methods for prediction and control?

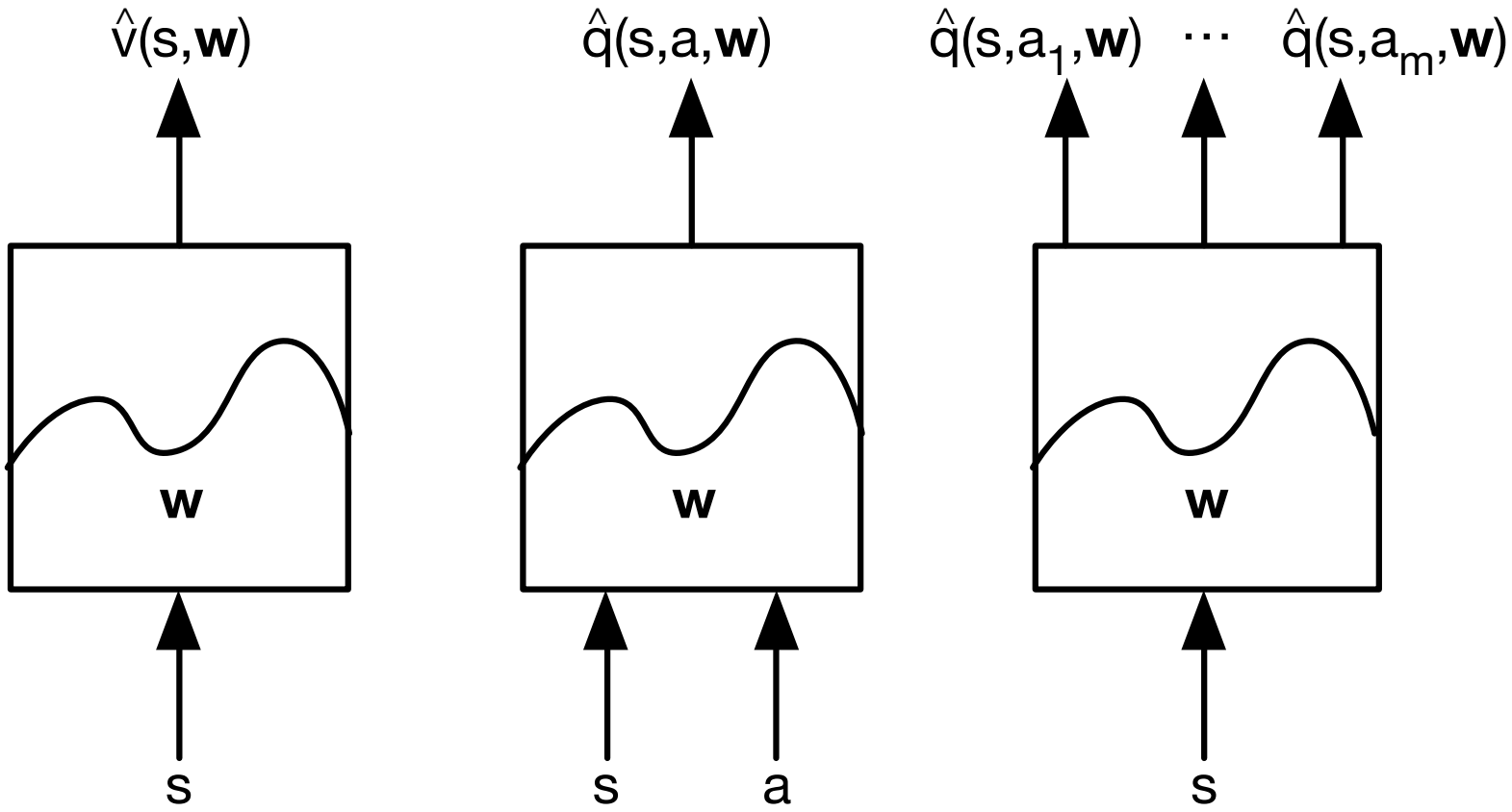
Value Function Approximation

- So far we have represented value function by a *lookup table*
  - Every state s has an entry $V(s)$
  - Or every state-action pair s, a has an entry Q(s,a)
- Problem with large MDPs:
  - There are too many states and/or actions to store in memory
  - It is too slow to learn the value of each state individually
- Solution for large MDPs:
  - Estimate value function with *function approximation*

$$\hat{v}(s, \mathbf{w}) \approx v_\pi(s)$$
$$\text{or } \hat{q}(s, a, \mathbf{w}) \approx q_\pi(s, a)$$

  - Update parameter $\mathbf{w}$ using MC or TD learning

Types of Value Function Approximation



Which Function Approximator?

There are many function approximators, and we consider differentiable of them, e.g.

- Linear combinations of features
- Neural network
- Decision tree
- Nearest neighbour
- Fourier / wavelet bases
- ...

Action-Value Function Approximation

- Approximate the action-value function

$$\hat{q}(S, A, \mathbf{w}) \approx q_\pi(S, A)$$

- Minimise mean-squared error between approximate action-value fn $\hat{q}(S, A, \mathbf{w})$ and true action-value fn $q_\pi(S, A)$

$$J(\mathbf{w}) = \mathbb{E}_\pi[(q_\pi(S, A) - \hat{q}(S, A, \mathbf{w}))^2]$$

- Use stochastic gradient descent to find a local minimum

$$-\frac{1}{2}\nabla_{\mathbf{w}} J(\mathbf{w}) = (q_\pi(S, A) - \hat{q}(S, A, \mathbf{w}))\nabla_{\mathbf{w}}\hat{q}(S, A, \mathbf{w})$$

$$\Delta\mathbf{w} = \alpha(q_\pi(S, A) - \hat{q}(S, A, \mathbf{w}))\nabla_{\mathbf{w}}\hat{q}(S, A, \mathbf{w})$$

Linear Action-Value Function Approximation

- Represent state and action by a feature vector

$$\mathbf{x}(S, A) = \begin{pmatrix} \mathbf{x}_1(S, A) \\ \vdots \\ \mathbf{x}_n(S, A) \end{pmatrix}$$

- Represent action-value fn by linear combination of features

$$\hat{q}(S, A, \mathbf{w}) = \mathbf{x}(S, A)^\top \mathbf{w} = \sum_{j=1}^{n} \mathbf{x}_j(S, A)\mathbf{w}_j$$

- Stochastic gradient descent update

$$\nabla_{\mathbf{w}} \hat{q}(S, A, \mathbf{w}) = \mathbf{x}(S, A)$$

$$\Delta \mathbf{w} = \alpha(q_\pi(S, A) - \hat{q}(S, A, \mathbf{w}))\mathbf{x}(S, A)$$

Incremental Control Algorithms

- Like prediction, we must substitute a target for $q_\pi(S, A)$
  - For MC, the target is the return G_t

$$\Delta \mathbf{w} = \alpha(G_t - \hat{q}(S_t, A_t, \mathbf{w}))\nabla_{\mathbf{w}}\hat{q}(S_t, A_t, \mathbf{w})$$

  - For TD(0), the target is the TD target $R_{t+1} + \gamma Q(S_{t+1}, A_{t+1})$

$$\Delta \mathbf{w} = \alpha(R_{t+1} + \gamma\hat{q}(S_{t+1}, A_{t+1}, \mathbf{w}) - \hat{q}(S_t, A_t, \mathbf{w}))\nabla_{\mathbf{w}}\hat{q}(S_t, A_t, \mathbf{w})$$

Outline

1 What is Reinforcement Learning?

2 Markov Decision Process

3 Value Based Methods
- Dynamic Programming
- Q-Learning
- Value Function Approximation

4 Policy Gradient Methods
- Likelihood Ratio Gradient
- Variance Reduction
- Actor Critic

5 Summary

Policy-Based Reinforcement Learning

- In Value-based method, a policy is generated directly from state-action value function Q(s,a)

$$a = \arg\max_a Q(s, a)$$

- We can also directly parametrise the policy

$$\pi_\theta(s, a) = P(a|s; \theta)$$

Why Policy Gradient?

Value-based method:

- need to be able to solve $\arg\max_a Q_\theta(s, a)$
- is inefficient for continuous/high dimensional action spaces

Policy-based method:

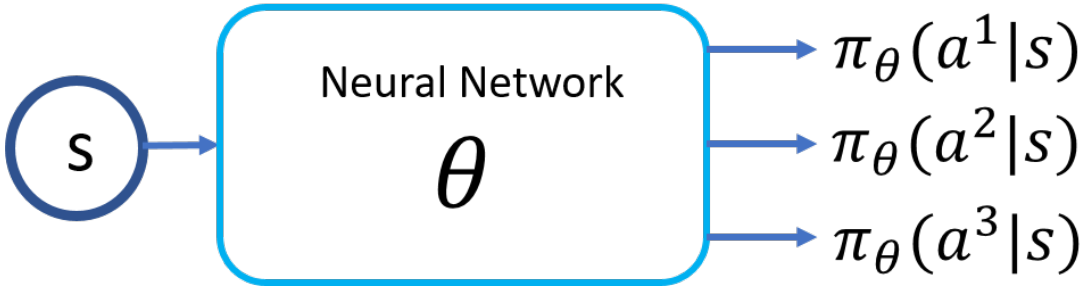
- can learn stochastic policies
- is effective in high-dimensional or continuous action spaces

Softmax Policy: Discrete Action

- Weight actions using linear combination of features $\Phi(s,a)^T\theta$
- Probability of action is proportional to exponentiated weight

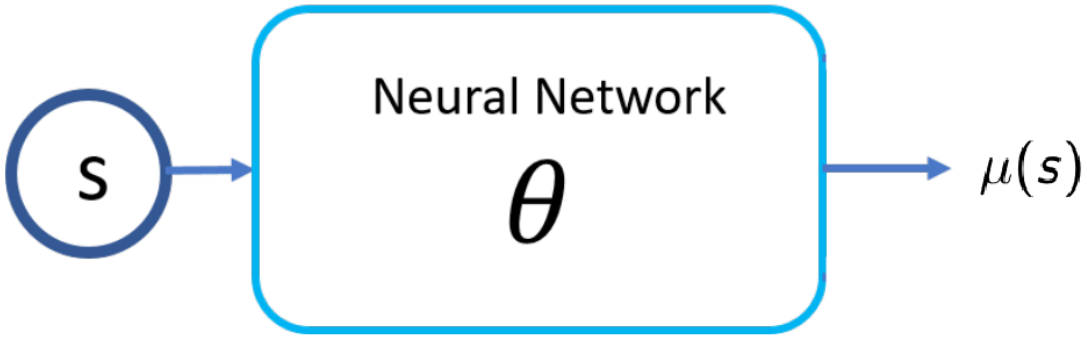
$$\pi_\theta(s,a) \propto e^{\Phi(s,a)^T\theta}$$

- Neural network combination



Gaussian Policy: Continuous action

- In continuous action space, a Gaussian policy is natural
- Mean is a linear combination of state features $\mu(s) = \Phi(s)^T\theta$
- Variance may be fixed σ^2, or can also parametrised
- Policy is Gaussian, $a \sim \mathcal{N}(\mu(s), \sigma^2)$
- Neural network combination



Objective Function

To measure the quality of a policy π_θ, we define the objective function:

$$J(\theta) = \sum_{\tau} P(\tau;\theta) R(\tau)$$

where τ denotes a state-action sequence, $\tau = (s_0, a_0, r_0, s_1, \ldots)$.

- $R(\tau)$ is the discounted reward of τ,

$$R(\tau) = \sum_{t=0}^{T-1} \gamma^t r_t$$

- $P(\tau;\theta)$ is the probability of τ,

$$P(\tau;\theta) = p(s_0) \prod_{t=0}^{T} \pi_\theta(s_t, a_t) p(s_{t+1}|s_t, a_t)$$

Gradient

Our goal is to find θ,

$$\max_{\theta} J(\theta) = \max_{\theta} \sum_{\tau} P(\tau;\theta)R(\tau)$$

Taking the gradient w.r.t. θ gives

$$\begin{aligned}\nabla_\theta J(\theta) &= \sum_{\tau} \nabla_\theta P(\tau;\theta)R(\tau) \\ &= \sum_{\tau} P(\tau;\theta)\frac{\nabla_\theta P(\tau;\theta)}{P(\tau;\theta)}R(\tau) \\ &= \sum_{\tau} P(\tau;\theta)\nabla_\theta \log P(\tau;\theta)R(\tau)\end{aligned}$$

Approximate the gradient,

$$\nabla_\theta J(\theta) \approx \frac{1}{N}\sum_{i=0}^{N} \nabla_\theta \log P(\tau^{(i)};\theta)R(\tau^{(i)})$$

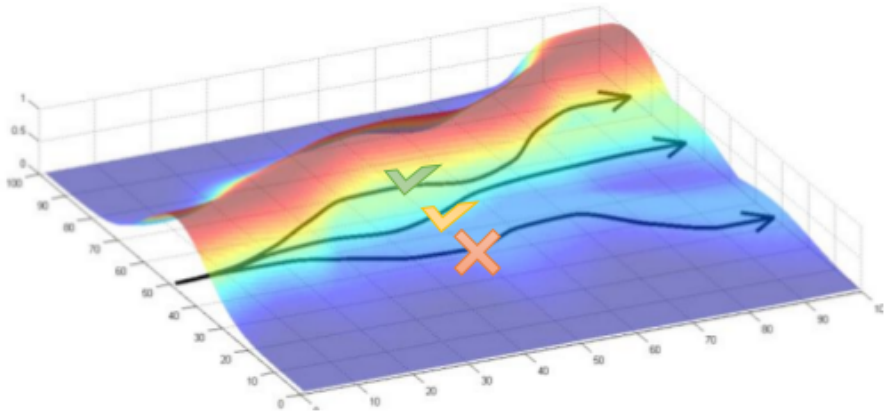
Gradient Intuition

Gradient estimator:

$$\nabla_\theta J(\theta) \approx \frac{1}{N} \sum_{i=0}^{N} \nabla_\theta \log P(\tau^{(i)}; \theta) R(\tau^{(i)})$$

Gradient tries to

- increase probability of paths with positive R
- decrease probability of paths with negative R



Decompose Path into States and Actions

$$
\begin{aligned}
\nabla_\theta \log P(\tau^{(i)};\theta) &= \nabla_\theta \log \left[\prod_{t=0}^{T} \underbrace{p(s_{t+1}^{(i)}|s_t^{(i)}, a_t^{(i)})}_{\text{dynamics model}} \cdot \underbrace{\pi_\theta(a_t^{(i)}|s_t^{(i)})}_{\text{policy}} \right] \\
&= \nabla_\theta \left[\sum_{t=0}^{T} \log p(s_{t+1}^{(i)}|s_t^{(i)}, a_t^{(i)}) + \sum_{t=0}^{T} \log \pi_\theta(a_t^{(i)}|s_t^{(i)}) \right] \\
&= \nabla_\theta \sum_{t=0}^{T} \log \pi_\theta(a_t^{(i)}|s_t^{(i)}) \\
&= \sum_{t=0}^{T} \underbrace{\nabla_\theta \log \pi_\theta(a_t^{(i)}|s_t^{(i)})}_{\text{no dynamics model required}}
\end{aligned}
$$

Gradient Estimate

The expression provides us with an unbiased estimate of the gradient, and we can compute it without access to a dynamics model,

$$\begin{aligned}\nabla_\theta J(\theta) &\approx \frac{1}{N}\sum_{i=1}^{N} R(\tau^{(i)}) \sum_{t=0}^{T} \nabla_\theta \log \pi_\theta(a_t^{(i)}|s_t^{(i)}) \\ &= \frac{1}{N}\sum_{i=1}^{N} \left(\sum_{t=0}^{T} \gamma^t r(s_t^{(i)}, a_t^{(i)})\right) \left(\sum_{t=0}^{T} \nabla_\theta \log \pi_\theta(a_t^{(i)}|s_t^{(i)})\right)\end{aligned}$$

High Variance

- A small change of reward can cause great changes on policy.

Reducing Variance

Causality: policy at time t' can not affect reward at time t when $t < t'$.

$$\begin{aligned}
\nabla_\theta J(\theta) &\approx \frac{1}{N}\sum_{i=1}^{N}\left(\sum_{t=0}^{T}\nabla_\theta \log \pi_\theta(a_{i,t}|s_{i,t})\right)\left(\sum_{t=0}^{T}\gamma^t r(s_{i,t}, a_{i,t})\right) \\
&= \frac{1}{N}\sum_{i=1}^{N}\sum_{t=0}^{T}\nabla_\theta \log \pi_\theta(a_{i,t}|s_{i,t})\underbrace{\sum_{t'=t}^{T}\gamma^{t'-t} r(s_{i,t'}, a_{i,t'})}_{\text{reward to go}} \\
&= \frac{1}{N}\sum_{i=1}^{N}\sum_{t=0}^{T}\nabla_\theta \log \pi_\theta(a_{i,t}|s_{i,t}) Q(s_{i,t}, a_{i,t})
\end{aligned}$$

The above estimator is still unbiased but with lower variance.

Reducing Variance

Baseline:

$$\nabla_\theta J(\theta) \approx \frac{1}{N} \sum_{i=1}^{N} \sum_{t=0}^{T} \nabla_\theta \log \pi_\theta(a_{i,t}|s_{i,t}) \left(Q(s_{i,t}, a_{i,t}) - b\right)$$

It is unbiased, for

$$\begin{aligned}
\mathbb{E}\left[\nabla_\theta \log P(\tau;\theta) b\right] &= \sum_\tau P(\tau;\theta) \nabla_\theta \log P(\tau;\theta) b \\
&= \sum_\tau \nabla_\theta P(\tau;\theta) b \\
&= \nabla_\theta \left(\sum_\tau P(\tau;\theta) b \right) \\
&= 0
\end{aligned}$$

Reducing Variance

- A good baseline is the average Q,

$$b_t = \frac{1}{N} \sum_i Q(s_{i,t}, a_{i,t})$$

$$V^{\pi_\theta}(s_t) = \mathbb{E}_{a_t \sim \pi_\theta(a_t|s_t)} \left[Q(s_t, a_t) \right]$$

- So we can rewrite the policy gradient using the advantage function $A^{\pi_\theta}(s, a)$,

$$A^{\pi_\theta}(s, a) = \underbrace{Q^{\pi_\theta}(s, a) - V^{\pi_\theta}(s)}_{\text{how much better } a \text{ is}}$$

Value Function Fitting

- Gradient estimator:

$$\nabla_\theta J(\theta) \approx \frac{1}{N} \sum_{i=1}^{N} \sum_{t=1}^{T} \nabla_\theta \log \pi_\theta(s_{i,t}, a_{i,t})(Q^{\pi_\theta}(s_{i,t}, a_{i,t}) - V^{\pi_\theta}(s_{i,t}))$$

- Estimate value function,

$$V^{\pi_\theta}(s_t) \approx \sum_{t'=t}^{T} \gamma^{t'-t} r(s_{t'}, a_{t'})$$

$$Q^{\pi_\theta}(s_t, a_t) = r + \gamma V^{\pi_\theta}(s_{t+1})$$

- Rollout estimate is inefficient.

Estimating the Advantage Function(1)

- Estimate both $Q^{\pi_\theta}(s, a)$ and $V^{\pi_\theta}(s)$
- Use two function approximators,

$$V_v(s) \approx V^{\pi_\theta}(s)$$

$$Q_w(s, a) \approx Q^{\pi_\theta(s,a)}$$

$$A(s, a) = Q_w(s, a) - V_v(s)$$

where v and w denote parameters of approximators.

Estimating the Advantage Function(2)

- For the true value function $V^{\pi_\theta}(s)$, the TD error δ^{π_θ}

$$\delta^{\pi_\theta} = r + \gamma V^{\pi_\theta}(s') - V^{\pi_\theta}(s)$$

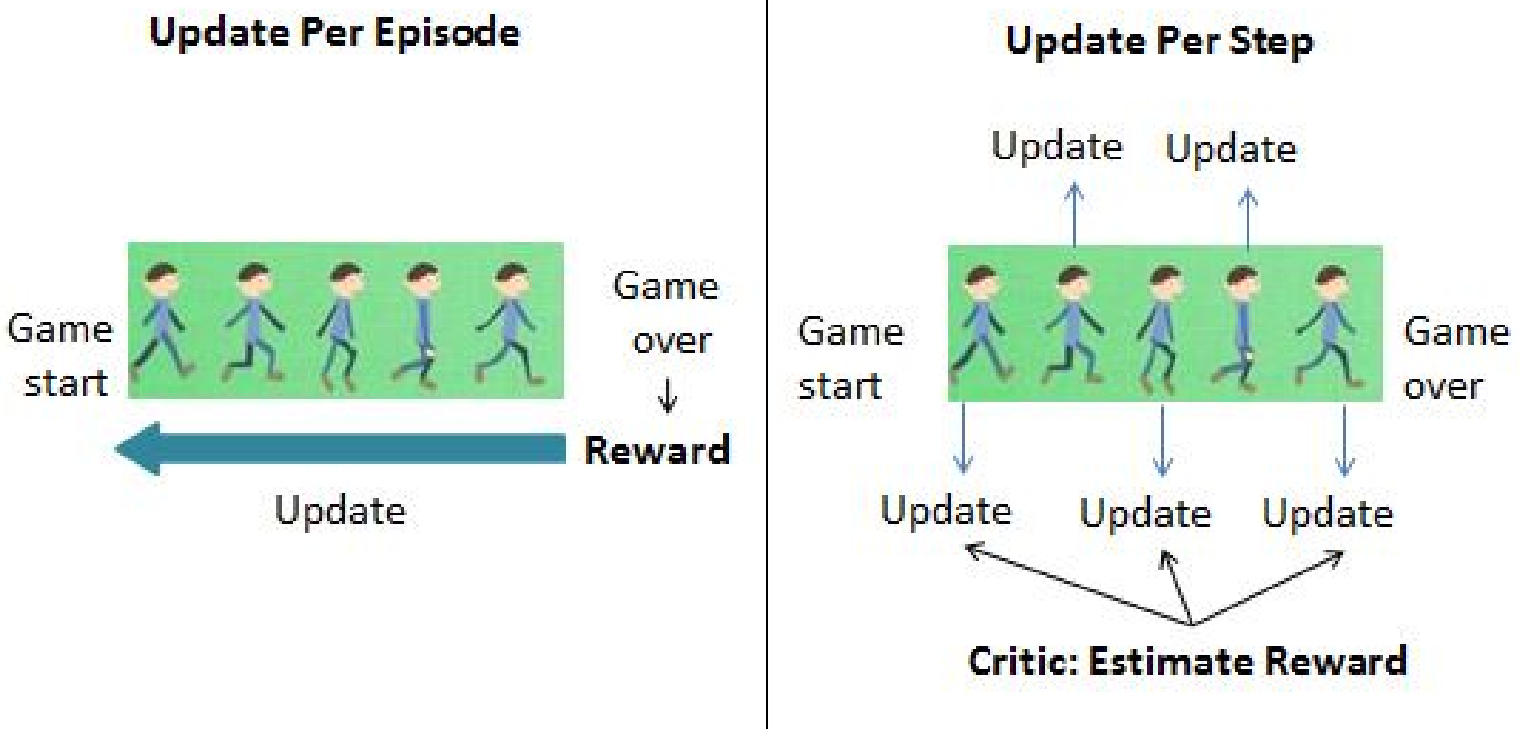
- δ^{π_θ} is an unbiased estimate of the advantage function

$$\begin{aligned}\mathbb{E}\left[\delta^{\pi_\theta}|s,a\right] &= \mathbb{E}_{\pi_\theta}\left[r + \gamma V^{\pi_\theta}(s')|s,a\right] - V^{\pi_\theta}(s) \\ &= Q^{\pi_\theta}(s,a) - V^{\pi_\theta}(s) \\ &= A^{\pi_\theta}(s,a)\end{aligned}$$

- So we can use the TD error to compute the policy gradient

$$\nabla_\theta J(\theta) \approx \frac{1}{N}\sum_{i=1}^{N}\sum_{t=1}^{T}\nabla_\theta \log \pi_\theta(s_{i,t}, a_{i,t})\delta_{i,t}^{\pi_\theta}$$

Actor Critic



- Use two function approximators
  - use actor $\pi_\theta(s, a)$ to decide which action to take
  - use critic $V(s; w)$ to evaluate the action

Actor-Critic Algorithm

Initialize actor $\pi_\theta(s, a)$ and critic $V(s; w)$
For trajectory=1,...,M do:
  Initialize s_0
  Repeat until s_i is terminal state:
    Take action a_i in state s_i according to π_θ
    Observe r_i, s_i'
    Compute TD error: $\delta = r_i + \gamma V(s_i'; w) - V(s_i; w)$
    Update critic: $\Delta w = \delta \nabla_w V(s_i; w)$
    Update actor:
    $\Delta\theta = \big(r_i + \gamma V(s_i'; w) - V(s_i; w)\big) \nabla_\theta \log \pi_\theta(a_i|s_i)$
    $s_{i+1} \leftarrow s_i'$
End for

Outline

1 What is Reinforcement Learning?

2 Markov Decision Process

3 Value Based Methods
- Dynamic Programming
- Q-Learning
- Value Function Approximation

4 Policy Gradient Methods
- Likelihood Ratio Gradient
- Variance Reduction
- Actor Critic

5 Summary

Categorizing RL agents (1)

- Value Based
  - No Policy (Implicit)
  - Value Function
- Policy Based
  - Policy
  - No Value Function
- Actor Critic
  - Policy
  - Value Function

Categorizing RL agents (2)

- Model Free
  - Policy and/or Value Function
  - No Model
- Model Based
  - Policy and/or Value Function
  - Model

References

- Silver, D.(2015). UCL Course on RL.
- Levine, S.(2017). CS 294: Deep Reinforcement Learning.
- Sutton, Richard S., and Andrew G. Barto. Reinforcement learning: An introduction. MIT press, 1998.

Any Questions?